Digital Information Processing Lab 2025

Class 3: Vocoder and STFT

Group 14 : Snoopy

08.06.2025

Name	Mat. Nr.	Email
Yash Khiste	254751	yash.khiste@st.ovgu.de
Saiful Islam	253574	saiful1.islam@st.ovgu.de
Supritha Rajashekaraiah	244941	supritha.rajashekaraiah@ovgu.de

Preparatory task

1. Define the Short Time Fourier Transform (STFT) and explain its purpose in signal processing. How does it differ from the regular Fourier Transform?

The Short-Time Fourier Transform (STFT) is a technique used to analyze the frequency content of local sections of a signal as it evolves over time. A fourier transform to short, overlapping time windows of the signal is applied. Mathematically, for a discrete-time signal

$$\mathrm{STFT}\{x[n]\}(m,\omega) = \sum_{n=-\infty}^{\infty} x[n] \cdot w[n-m] \cdot e^{-j\omega n}$$

where:

```
x[n] is the signal,
w[n-m] is the window function centered at time index m
ω is the frequency variable.
```

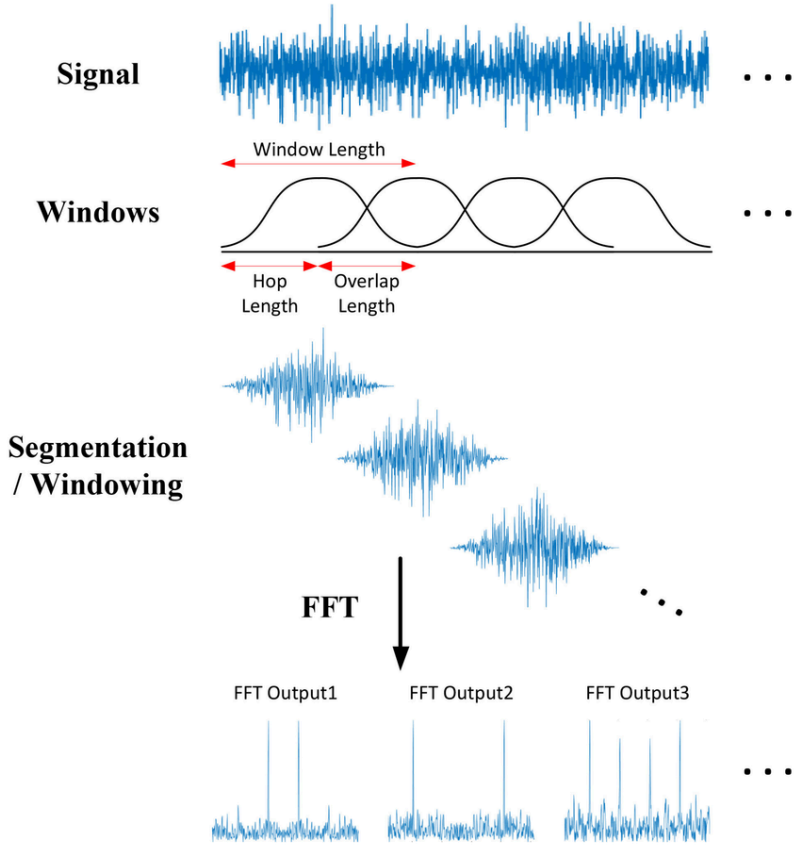


Fig_1: Short-time Fourier transform (STFT) overview. [1]

Purpose in Signal Processing:

STFT allows us to observe how the frequency and amplitude components of a signal change over time, which is especially important for analyzing non-stationary signals.

Difference from the Regular Fourier Transform:

Fourier Transform (FT) provides frequency content for the entire signal frequency components in the signal averaged over all time, assuming it is stationary.

STFT provides frequency content over time-localized windows, thus capturing time-varying behavior.

2. Discuss the concept of time-frequency analysis and explain why it is useful in analyzing non-stationary signals.

Time–frequency analysis is a signal processing technique that examines signals in both the time and frequency domains simultaneously. This approach is particularly beneficial for analysing non-stationary signals—those whose frequency content changes over time.

Usefulness:

1.Many real-world signals are non-stationary, meaning their statistical properties change over time. For instance, musical notes vary in pitch and duration, and speech signals fluctuate rapidly as different phonemes are articulated. Time–frequency analysis is crucial for capturing these dynamics, as it allows for the detection of transient events, pitch variations, and modulations that traditional Fourier analysis might overlook.

2.The Fourier analysis provides information about the frequency components of a signal but lacks temporal resolution; it doesn't indicate when specific frequencies occur. Time–frequency analysis addresses this limitation by representing a signal as a two-dimensional function over time and frequency, offering a more comprehensive view of how the signal's spectral content evolves. This is achieved through various time–frequency representations, which transform a one-dimensional signal into a two-dimensional function via time–frequency transforms.

3. What is the spectrogram and how is it related to the STFT? Explain how the spectrogram can be used to visualize time-varying frequency content in a signal.

A spectrogram is a visual representation of the magnitude of the STFT over time. It Plots as a 2D image: time on the x-axis, frequency on the y-axis, and magnitude (intensity) shown with color or brightness.

$$\text{Spectrogram}(m, \omega) = |\text{STFT}(m, \omega)|^2$$

Use:

1. Provides 2D head map showing how freqency and amplitude vary over time.
2. Useful in real-world applications such as music/audio analysis and processing, biomedical signal analysis, fault detection, and vibration analysis.

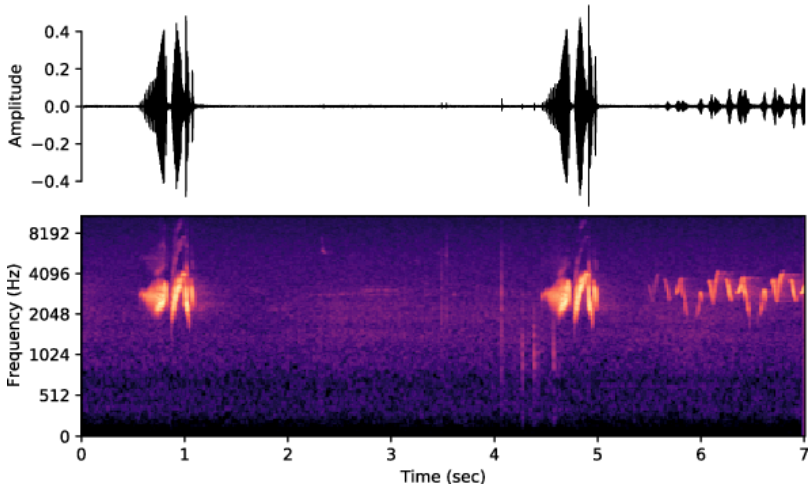


Fig_1: Audio spectrogram representation. The raw audio signal is transformed using the Fourier transform into a mel-spectrogram image. The frequency on the y-axis is in the mel scale. [2]

Relation to STFT:

1. It's derived directly from the STFT.
2. It discards the phase information and shows only the squared magnitude.

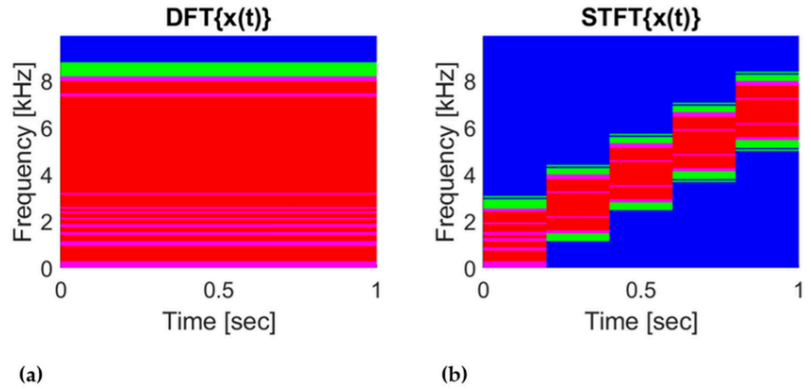


Fig_1: (a) spectrogram of FFT of the entire signal, with no information along time; (b) spectrogram of STFT of the same time signal with data variation due to the STFT process. The colors express power, with a high number being red, a medium number being green and a low number being blue. [3]

4. What is the Phase Vocoder and what is its role in time stretching or pitch shifting audio signals? Explain the basic principles behind the Phase Vocoder algorithm.

The Phase Vocoder is a signal processing technique that uses the STFT to modify the duration (time-stretch) or frequency (pitch-shift) of a signal without affecting the other.

Basic Principles:

1. STFT Analysis: Signal is divided into overlapping frames, and STFT is applied.

Phase Unwrapping: Phase continuity is maintained across frames to preserve signal integrity.

2. Modification:

- For time-stretching, frames are spaced differently during synthesis.
- For pitch-shifting, frequency bins are resampled or transposed.

3. STFT Synthesis: The inverse STFT reconstructs the time-domain signal with modified timing or pitch.

This process relies heavily on accurate phase information. [5]

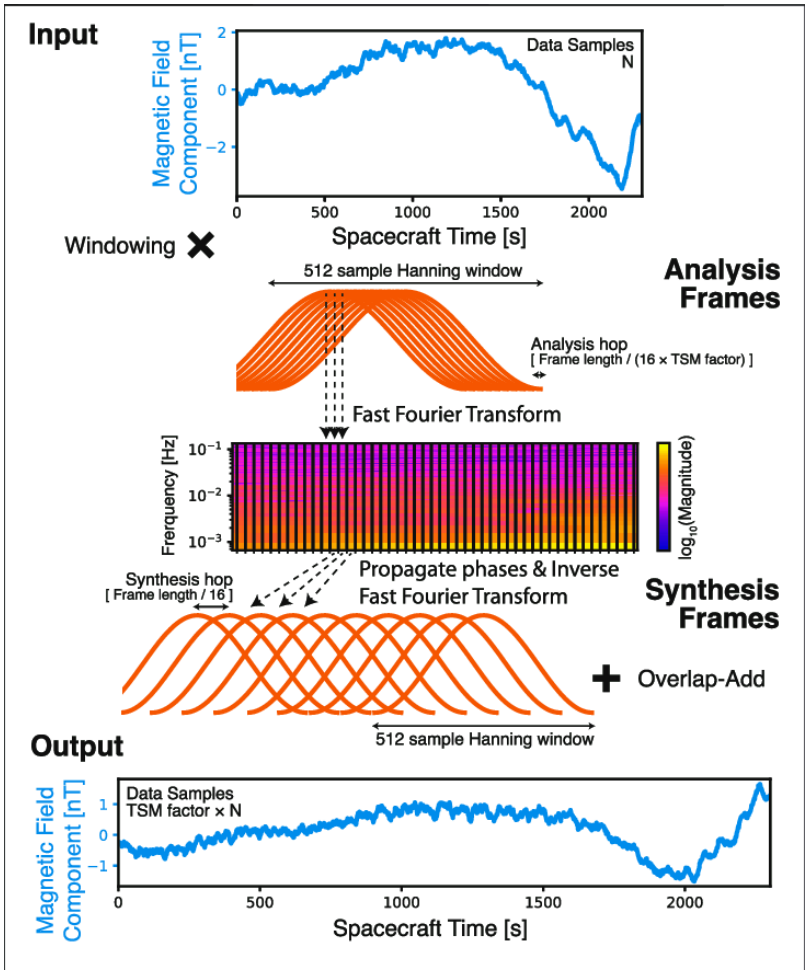


Fig_4: Illustration of the Phase Vocoder method. [4]

5. Discuss the limitations of the Phase Vocoder and possible artifacts that can occur during time stretching or pitch shifting.

Phase Vocoder Equations

1. Phase deviation:

$$\Delta\phi_k = \phi_k(n) - \phi_k(n-R) - \omega_k R$$

2. True frequency estimate:

$$\hat{\omega}_k = \omega_k + \frac{\Delta\phi_k}{R}$$

3. Phase update for synthesis:

$$\phi_k^{\text{new}}(n) = \phi_k^{\text{new}}(n - R') + \hat{\omega}_k R'$$

Limitations:

- Assumes quasi-stationarity within each window.
- Accuracy depends on the hop size, window length, and phase estimation.
- Works best for harmonic, periodic signals; less effective for transient or percussive sounds.

Common Artifacts (from Oppenheim & Schafer and practice):

- Phasiness / Smearing: Loss of clarity due to incorrect phase interpolation.
- Transient Blurring: Sharp events (e.g., drum hits) become smeared or dull.
- Echoes or Reverberation: Especially when time-stretched excessively.
- Pitch Artifacts: In pitch shifting, non-harmonic content may produce warbled or metallic sounds. [5]

Class 3: Vocoder and STFT

Exercise 1. Task 1: Short Time Fourier Transform (STFT)

In this task, you will explore the Short Time Fourier Transform (STFT) and its application in analyzing audio signals. The objective is to gain a better understanding of the frequency content of an audio signal over time. Follow the steps below:

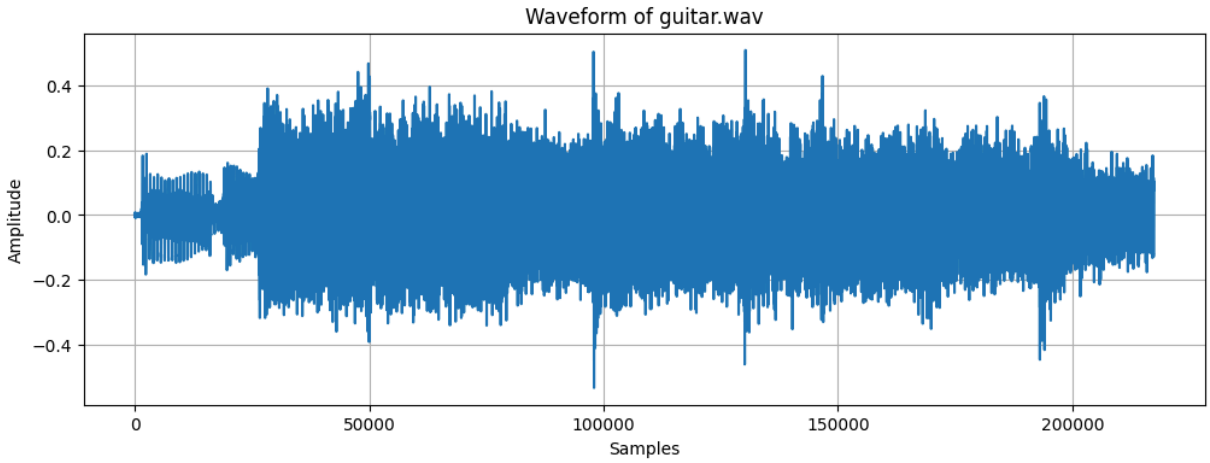
- Load the provided 'guitar.wav' audio file into Google Colab.
- Visualize the waveform of the audio file to get an overview of the signal.
- Apply the STFT to the audio signal using an appropriate window size and overlap. Experiment with different parameters to observe their effects.
- Display the resulting spectrogram, which represents the frequency content of the audio signal over time.
- Explore the spectrogram and analyze the changes in the frequency content of the audio signal.
- Modify the window size and overlap and observe how they affect the resolution and clarity of the spectrogram.
- Apply the inverse STFT to the modified spectrogram to obtain the reconstructed audio signal.
- Compare the reconstructed audio signal with the original audio to evaluate the quality of reconstruction. **Explain what is happening and why**.

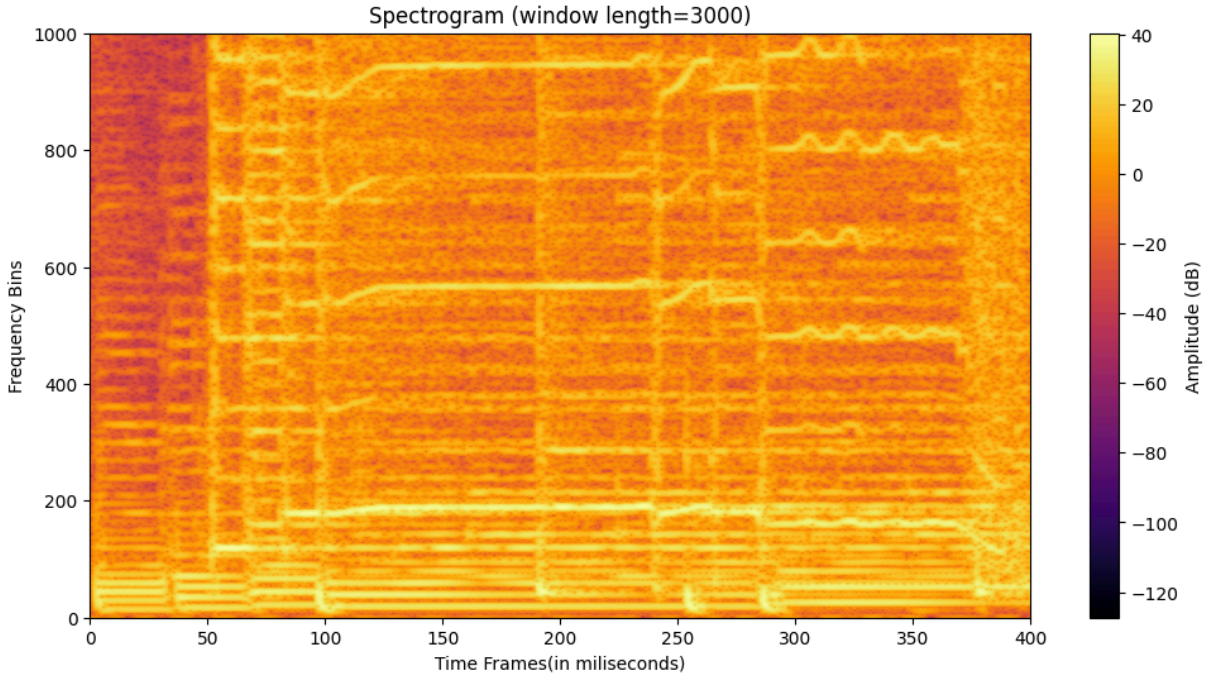
In [1]:
```
!pip install torchaudio gdown

import torch
import torchaudio
import torchaudio.transforms as T
import matplotlib.pyplot as plt
import numpy as np
import IPython.display as ipd

# =====================
# Download guitar.wav from Google Drive
# =====================
# File ID from the provided link: https://drive.google.com/file/d/1OFdiFNftyzMlgfi6h96ZAahBDEYaLqV9/view
!gdown 1OFdiFNftyzMlgfi6h96ZAahBDEYaLqV9

# =====================
# Load the audio
# =====================
waveform, sample_rate = torchaudio.load("guitar.wav")
print(f"Waveform shape: {waveform.shape}, Sample rate: {sample_rate}")

# =====================
# Visualize the waveform
# =====================
```

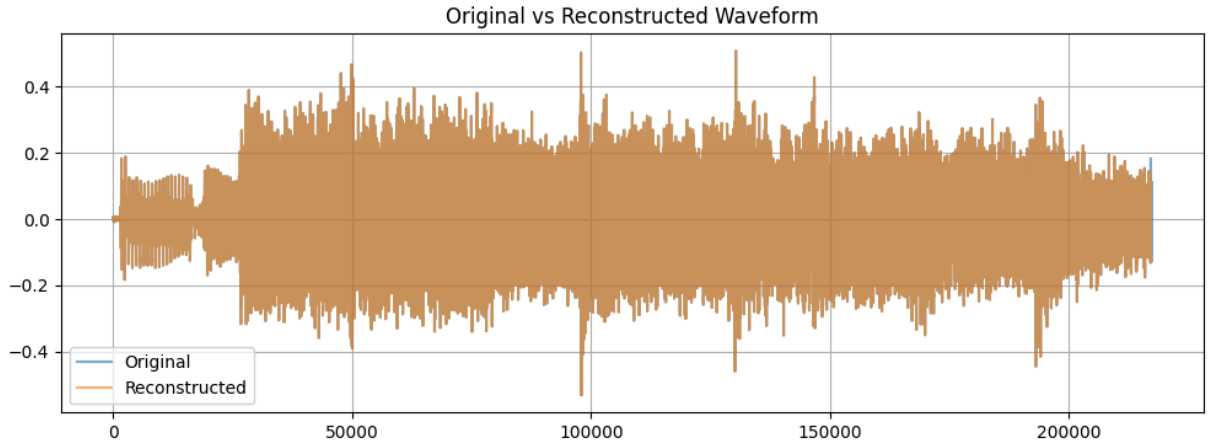
```
plt.figure(figsize=(12, 4))
plt.plot(waveform.t().numpy())
plt.title("Waveform of guitar.wav")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid()
plt.show()

# =====================
# Apply STFT and show spectrogram
# =====================
n_fft = 12000
hop_length = 512
win_length = 3000

transform = T.Spectrogram(n_fft=n_fft, hop_length=hop_length, win_length=win_length, power=None)
spectrogram = transform(waveform)

# Convert to dB for visualization
magnitude_db = 20 * torch.log10(spectrogram.abs() + 1e-10)

plt.figure(figsize=(12, 6))
plt.imshow(magnitude_db[0].numpy(), aspect='auto', origin='lower', cmap='inferno')
plt.title("Spectrogram (window length=3000)")
plt.xlabel("Time Frames(in miliseconds)")
plt.xlim(0, 400)
plt.ylabel("Frequency Bins")
plt.ylim(0, 1000)
plt.colorbar(label="Amplitude (dB)")
plt.show()


# =====================
# Inverse STFT to reconstruct audio
# =====================
inverse_transform = T.InverseSpectrogram(n_fft=n_fft, hop_length=hop_length, win_length=win_length)
reconstructed_waveform = inverse_transform(spectrogram)

# =====================
# Compare Original and Reconstructed Waveform
# =====================
plt.figure(figsize=(12, 4))
plt.plot(waveform[0].numpy(), label="Original", alpha=0.6)
plt.plot(reconstructed_waveform[0].numpy(), label="Reconstructed", alpha=0.6)
plt.title("Original vs Reconstructed Waveform")
plt.legend()
plt.grid()
plt.show()

# =====================
# Listen to the audio
# =====================
print(" Original Audio:")
ipd.display(ipd.Audio(waveform.numpy(), rate=sample_rate))

print("Reconstructed Audio:")
ipd.display(ipd.Audio(reconstructed_waveform.numpy(), rate=sample_rate))
```

```
Requirement already satisfied: torchaudio in /usr/local/lib/python3.11/dist-packages (2.6.0+cu124)
Requirement already satisfied: gdown in /usr/local/lib/python3.11/dist-packages (5.2.0)
Requirement already satisfied: torch==2.6.0 in /usr/local/lib/python3.11/dist-packages (from torchaudio)
(2.6.0+cu124)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->to
rchaudio) (3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from
torch==2.6.0->torchaudio) (4.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->to
rchaudio) (3.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torc
haudio) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torc
haudio) (2025.3.2)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.5.147 (from torch==2.6.0->torchaudio)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparse-cu12==12.3.1.170 (from torch==2.6.0->torchaudio)
  Downloading nvidia_cusparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in /usr/local/lib/python3.11/dist-packages
(from torch==2.6.0->torchaudio) (0.6.2)
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/python3.11/dist-packages (from
torch==2.6.0->torchaudio) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (fro
m torch==2.6.0->torchaudio) (12.4.127)
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch==2.6.0->torchaudio)
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.
0->torchaudio) (3.2.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.
0->torchaudio) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy=
=1.13.1->torch==2.6.0->torchaudio) (1.3.0)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.11/dist-packages (from gdown) (4.
13.4)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.11/dist-packages (from gdown)
(2.32.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from gdown) (4.67.1)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulso
up4->gdown) (2.7)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from
requests[socks]->gdown) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests[soc
ks]->gdown) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from reques
ts[socks]->gdown) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from reques
ts[socks]->gdown) (2025.6.15)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.11/dist-packages (from re
quests[socks]->gdown) (1.7.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->t
orch==2.6.0->torchaudio) (3.0.2)
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (363.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 363.4/363.4 MB 3.0 MB/s eta 0:00:00
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (13.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.8/13.8 MB 19.5 MB/s eta 0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (24.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.6/24.6 MB 14.6 MB/s eta 0:00:00
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 883.7/883.7 kB 15.9 MB/s eta 0:00:00
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 2.1 MB/s eta 0:00:00
```

```
Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (211.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 211.5/211.5 MB 4.8 MB/s eta 0:00:00
Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl (56.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.3/56.3 MB 11.0 MB/s eta 0:00:00
Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl (127.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 127.9/127.9 MB 7.4 MB/s eta 0:00:00
Downloading nvidia_cusparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl (207.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 207.5/207.5 MB 5.8 MB/s eta 0:00:00
Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (21.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21.1/21.1 MB 40.4 MB/s eta 0:00:00
Installing collected packages: nvidia-nvjitlink-cu12, nvidia-curand-cu12, nvidia-cufft-cu12, nvidia-cuda-
runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, nvidia-cusparse-cu12, n
vidia-cudnn-cu12, nvidia-cusolver-cu12
  Attempting uninstall: nvidia-nvjitlink-cu12
    Found existing installation: nvidia-nvjitlink-cu12 12.5.82
    Uninstalling nvidia-nvjitlink-cu12-12.5.82:
      Successfully uninstalled nvidia-nvjitlink-cu12-12.5.82
  Attempting uninstall: nvidia-curand-cu12
    Found existing installation: nvidia-curand-cu12 10.3.6.82
    Uninstalling nvidia-curand-cu12-10.3.6.82:
      Successfully uninstalled nvidia-curand-cu12-10.3.6.82
  Attempting uninstall: nvidia-cufft-cu12
    Found existing installation: nvidia-cufft-cu12 11.2.3.61
    Uninstalling nvidia-cufft-cu12-11.2.3.61:
      Successfully uninstalled nvidia-cufft-cu12-11.2.3.61
  Attempting uninstall: nvidia-cuda-runtime-cu12
    Found existing installation: nvidia-cuda-runtime-cu12 12.5.82
    Uninstalling nvidia-cuda-runtime-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-runtime-cu12-12.5.82
  Attempting uninstall: nvidia-cuda-nvrtc-cu12
    Found existing installation: nvidia-cuda-nvrtc-cu12 12.5.82
    Uninstalling nvidia-cuda-nvrtc-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.5.82
  Attempting uninstall: nvidia-cuda-cupti-cu12
    Found existing installation: nvidia-cuda-cupti-cu12 12.5.82
    Uninstalling nvidia-cuda-cupti-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-cupti-cu12-12.5.82
  Attempting uninstall: nvidia-cublas-cu12
    Found existing installation: nvidia-cublas-cu12 12.5.3.2
    Uninstalling nvidia-cublas-cu12-12.5.3.2:
      Successfully uninstalled nvidia-cublas-cu12-12.5.3.2
  Attempting uninstall: nvidia-cusparse-cu12
    Found existing installation: nvidia-cusparse-cu12 12.5.1.3
    Uninstalling nvidia-cusparse-cu12-12.5.1.3:
      Successfully uninstalled nvidia-cusparse-cu12-12.5.1.3
  Attempting uninstall: nvidia-cudnn-cu12
    Found existing installation: nvidia-cudnn-cu12 9.3.0.75
    Uninstalling nvidia-cudnn-cu12-9.3.0.75:
      Successfully uninstalled nvidia-cudnn-cu12-9.3.0.75
  Attempting uninstall: nvidia-cusolver-cu12
    Found existing installation: nvidia-cusolver-cu12 11.6.3.83
    Uninstalling nvidia-cusolver-cu12-11.6.3.83:
      Successfully uninstalled nvidia-cusolver-cu12-11.6.3.83
Successfully installed nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-cu12-12.4.127 nvidia-cuda-nvrtc-cu12
-12.4.127 nvidia-cuda-runtime-cu12-12.4.127 nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3 nvidia-
curand-cu12-10.3.5.147 nvidia-cusolver-cu12-11.6.1.9 nvidia-cusparse-cu12-12.3.1.170 nvidia-nvjitlink-cu1
2-12.4.127
Downloading...
From: https://drive.google.com/uc?id=1OFdiFNftyzMlgfi6h96ZAahBDEYaLqV9
To: /content/guitar.wav
100% 435k/435k [00:00<00:00, 77.9MB/s]
Waveform shape: torch.Size([1, 217364]), Sample rate: 44100
```







Original Audio:

0:00 / 0:04

Reconstructed Audio:

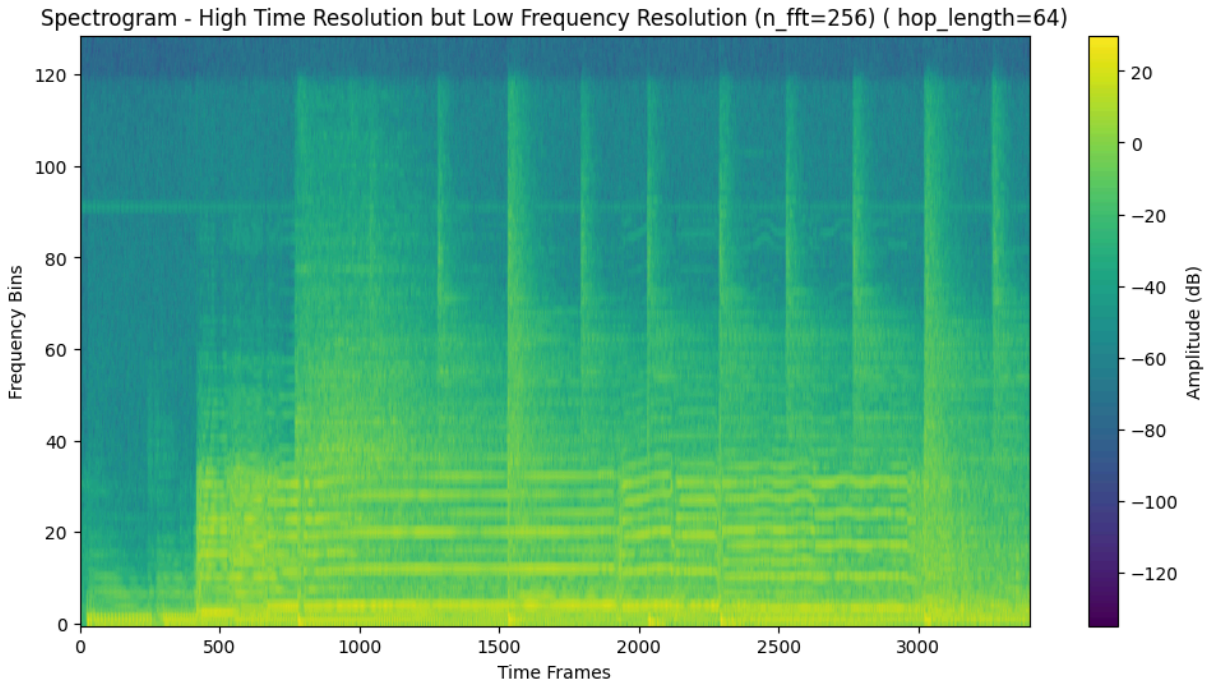
0:00 / 0:04

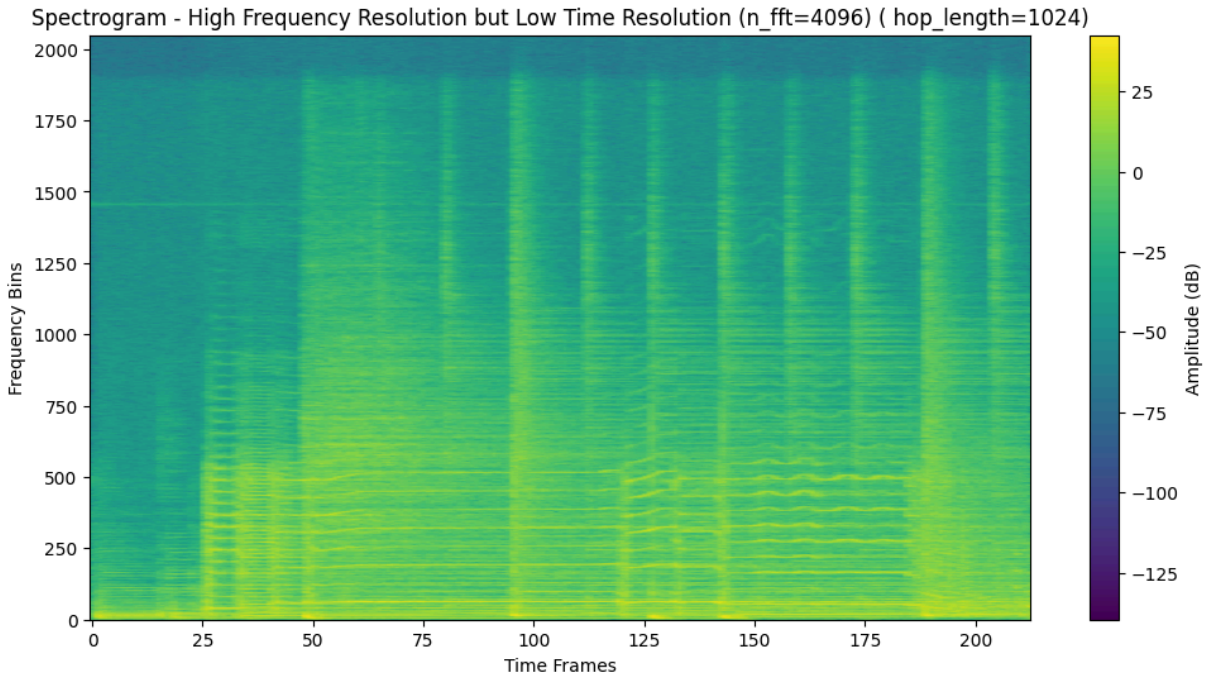
Explanation of what is happening.

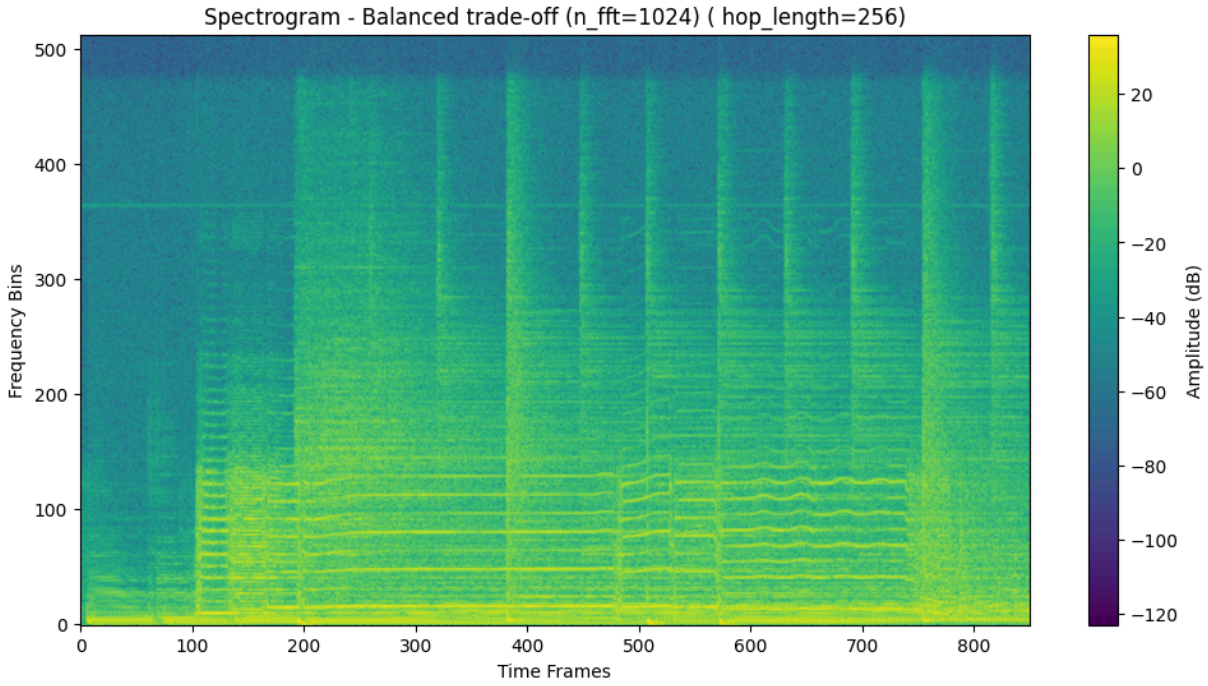
- The first graph shows the time-domain audio signal.

- STFT slices the audio into small overlapping windows and applies FFT to each, capturing how the frequency content changes over time.
- Window size (n_fft) controls frequency resolution: larger windows → finer frequency bins but worse time resolution.
- Hop length controls the step between windows: smaller hop → better time resolution (more frames), but more computation.
- The second graph shows the spectrogram of the original signal with window lenght = 3000 and hop_length = 512.
- Bright areas in the spectrogram indicate higher amplitude at certain frequencies and times.
- Inverse STFT reconstructs the time-domain audio from the spectrogram.
- The third graph shows the reconstructed signal in the time domain, which overlaps exactly with the original because of the correct window size and hop length.

In [2]:
```python
# Compare large vs small window
configs = [
    {"n_fft": 256, "hop_length": 64, "title": "High Time Resolution but Low Frequency Resolution"},
 {"n_fft": 4096, "hop_length": 1024, "title": "High Frequency Resolution but Low Time Resolution"},
 {"n_fft": 1024, "hop_length": 256, "title": "Balanced trade-off"},

]

for cfg in configs:
    transform = T.Spectrogram(
        n_fft=cfg["n_fft"],
        hop_length=cfg["hop_length"],
        win_length=cfg["n_fft"],
        power=None
    )
    spec = transform(waveform)
    mag_db = 20 * torch.log10(spec.abs() + 1e-10)

    plt.figure(figsize=(12, 6))
    plt.imshow(mag_db[0].numpy(), aspect='auto', origin='lower', cmap='viridis')
    plt.title(f"Spectrogram - {cfg['title']} (n_fft={cfg['n_fft']}) ( hop_length={cfg['hop_length']})")
    plt.xlabel("Time Frames")
    plt.ylabel("Frequency Bins")

    plt.colorbar(label="Amplitude (dB)")
    plt.show()
```







Modifying the window size and overlap length:

- n_fft : FFT window size. Affects frequency resolution. Higher = better frequency resolution, but poor time resolution and visa versa
- hop_length : Step between windows. Affects time resolution. Smaller = better time resolution, but high computation and visa versa.
- sample_rate: Number of samples per second. Affects overall time/frequency scale. (commonly 44,100 Hz).
- High time resolution : n_fft = 256 and hop_length = 64.

  Provides smoother transitions between frames. Better time resolution but increases computational load. Close frequecnies cannot be seen properly.
- High frequency resolution : n_fft = 4096 and hop_length = 1024.

  Improves frequency resolution (frequency bins are narrower). The spectrogram shows clearer, more detailed frequency components with harmonics as well. But blurs fast changes.

- Balanced : n_fft = 1024 and hop_length = 256. Balanced/moderate time and frequency resolution, While it's not the best at either extreme (like high-resolution pitch analysis or ultra-fast transients), it performs well enough in both area.

In [3]:
```
windows = {
    "hann": torch.hann_window,
    "hamming": torch.hamming_window,
    "blackman": torch.blackman_window,
    "Rectangular": torch.ones
}

n_fft = 3000
hop_length = 256

for name, win_fn in windows.items():
    window = win_fn(n_fft)
    transform = T.Spectrogram(
        n_fft=n_fft,
        hop_length=hop_length,
        win_length=n_fft,
        window_fn=lambda n_fft: window,
        power=None
    )
    spec = transform(waveform)
    mag_db = 20 * torch.log10(spec.abs() + 1e-10)

    plt.figure(figsize=(12, 6))
    plt.imshow(mag_db[0].numpy(), aspect='auto', origin='lower', cmap='plasma')
    plt.title(f"Spectrogram using {name.title()} Window")
    plt.xlabel("Time Frames")
    plt.ylabel("Frequency Bins")
    plt.xlim(0, 400)
    plt.ylim(0, 1000)

    plt.colorbar(label="Amplitude (dB)")
    plt.show()
```

```
Output hidden; open in https://colab.research.google.com to view.
```

Trade-Off between window size and overlapping:-

We can't maximize both time and frequency resolution at the same time.

Choose:

1. Using a small window size (e.g., n_fft = 256) provides high time resolution, capturing quick changes in the signal accurately, but has low frequency resolution, making frequencies less precise. (e.g., drums, speech).
2. Using a large window size (e.g., n_fft = 4096) gives high frequency resolution, meaning you can distinguish close frequencies better, but results in low time resolution, causing fast changes in the signal to be blurred.(e.g., violin, guitar chords).
3. Smaller hop lengths increase the number of frames, improving time resolution but also increasing computation time. By using the variable hop_lenght we can balance the resolution.

Window function

- Windowing controls spectral leakage, which occurs when sharp edges in the window introduce false frequency components.
- STFT multiplies the signal by a window function before FFT to capture frequecny and reduce edge effects.

We have done observation over 3 windows by their spectrogram at n_fft = 1024 and hop_length = 256

- The Hann window is most commonly used for STFT due to good trade-offs. We can see both frequency and time resolutions are good.
- Hamming window: Slightly better frequency resolution than Hann. Time resolutions is almost same as Hann.

- Blackman suppresses spectral leakage best but reduces resolution with minimal noise.
- Rectangular (no window) gives sharp time localization, but creates artifacts in frequency.

STFT Limitation:

STFT uses fixed window size throughout the signal.

It assumes stationarity within each window, which is not true for fast-changing audio. [1]

Limitation	Description	Impact
Fixed resolution	Same `n_fft` used throughout	Can't adapt to fast and slow changes
Time-frequency trade-off	Can't capture short and long events simultaneously	Must choose what detail to prioritize
Smearing of transients	Large windows blend events over time	Onsets and attacks become blurry
Poor for non-periodic signals	Doesn't represent time-varying pitch well	Glissandos and vibratos not cleanly visible

|Assumption of Stationarity Within Window | Non-stationary signals violate this assumption | inaccurate spectral representations

Exercise 2. Task 2: Pitch Correction using Phase Vocoder

In this task, you will explore pitch correction using the Phase Vocoder algorithm. The objective is to correct the pitch of a singer's performance to a target frequency, such as a "perfect" 440 Hz. Follow the steps below:

- Record a short audio clip of yourself holding a constant note.
- Load the audio clip into Google Colab and visualize its waveform.
- Apply the Phase Vocoder algorithm to modify the pitch of your performance, shifting it towards the target frequency (e.g., 440 Hz).
- Adjust the pitch correction factor to achieve the desired pitch correction effect.
- Play the modified audio and compare it with the original recording to evaluate the pitch correction.
- Analyze the spectrograms of the original and modified audio to observe the changes in the frequency content and pitch accuracy.

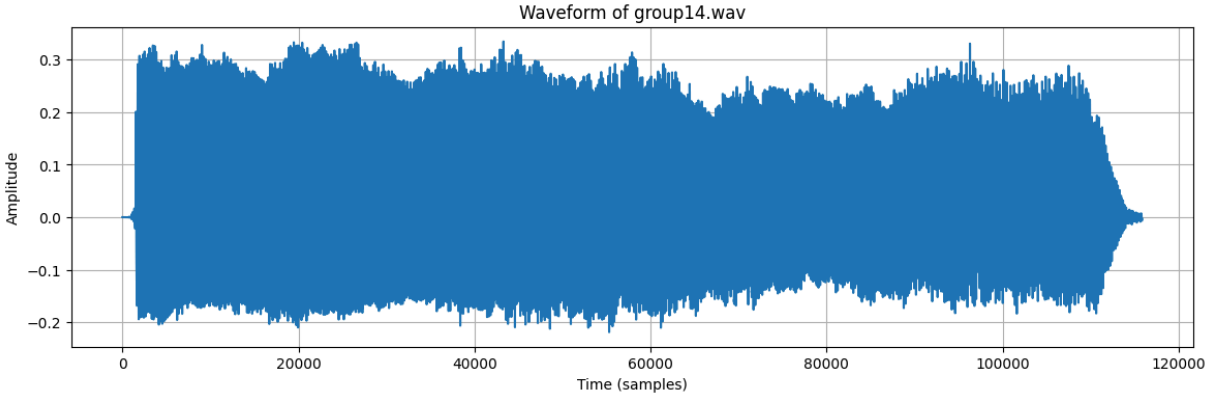
Take note of the following considerations:

```
- Experiment with different pitch correction settings to find the optimal balance
between naturalness and pitch accuracy.
- Reflect on the impact of pitch correction on the overall quality and perception of
your performance.
- Consider the limitations and challenges of pitch correction, and discuss potential
artifacts that may arise during the correction process.
```

In [4]:
```
# Install libraries
!pip install torchaudio matplotlib gdown

# Import required modules
import torchaudio
import matplotlib.pyplot as plt
import torchaudio.transforms as T
import gdown

# Define the file ID and construct direct download URL
file_id = "1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL"
url = f"https://drive.google.com/uc?id={file_id}"
output = "group14.wav"

# Download the file
gdown.download(url, output, quiet=False)

# Load the audio
waveform, sample_rate = torchaudio.load(output)
```

```
# Print waveform shape
print(f"Waveform shape: {waveform.shape}")
print(f"Sample rate: {sample_rate}")

# Plot the waveform
plt.figure(figsize=(14, 4))
plt.plot(waveform.t().numpy())
plt.title("Waveform of group14.wav")
plt.xlabel("Time (samples)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>
=2.7->matplotlib) (1.17.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulso
up4->gdown) (2.7)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from
requests[socks]->gdown) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests[soc
ks]->gdown) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from reques
ts[socks]->gdown) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from reques
ts[socks]->gdown) (2025.6.15)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.11/dist-packages (from re
quests[socks]->gdown) (1.7.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->t
orch==2.6.0->torchaudio) (3.0.2)
Downloading...
From: https://drive.google.com/uc?id=1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL
To: /content/gdrive/MyDrive/Colab Notebooks/group14.wav
  0%|          | 0.00/232k [00:00<?, ?B/s]
100%|██████████| 232k/232k [00:00<00:00, 38.8MB/s]
Waveform shape: torch.Size([1, 115848])
Sample rate: 48000
```



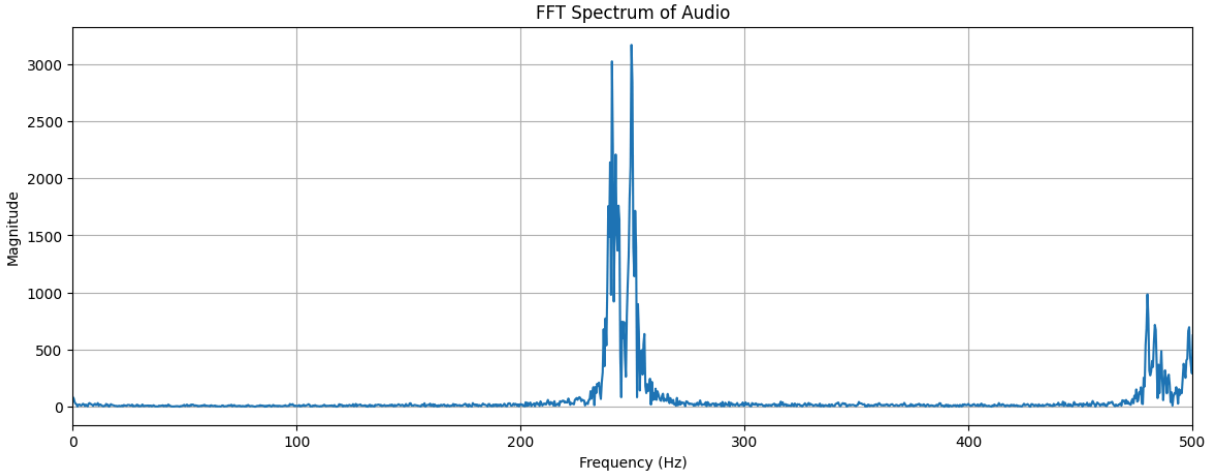
FFT Spectrum

In [2]:

```python
import torchaudio
import torchaudio.transforms as T
import matplotlib.pyplot as plt
import numpy as np
import gdown

# Download the audio file
file_id = "1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL"
url = f"https://drive.google.com/uc?id={file_id}"
output = "group14.wav"
gdown.download(url, output, quiet=False)

# Load and resample to 44100 Hz
waveform, sample_rate = torchaudio.load(output)
target_sr = 44100
if sample_rate != target_sr:
    resample = T.Resample(orig_freq=sample_rate, new_freq=target_sr)
    waveform = resample(waveform)

# Convert to NumPy
waveform_np = waveform.squeeze().numpy()

# FFT analysis
n = len(waveform_np)
fft_result = np.fft.fft(waveform_np)
fft_freq = np.fft.fftfreq(n, d=1/target_sr)
magnitude = np.abs(fft_result)[:n // 2]
freqs = fft_freq[:n // 2]

# Plot FFT spectrum
plt.figure(figsize=(14, 5))
```

```
plt.plot(freqs, magnitude)
plt.title("FFT Spectrum of Audio")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.xlim(0, 500)  # limit for voice analysis
plt.show()
```

```
Downloading...
From: https://drive.google.com/uc?id=1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL
To: /content/gdrive/MyDrive/Colab Notebooks/group14.wav
  0%|          | 0.00/232k [00:00<?, ?B/s]
100%|██████████| 232k/232k [00:00<00:00, 20.4MB/s]
```



Identification of closest note

In [3]:

```
import torchaudio
import torchaudio.transforms as T
import matplotlib.pyplot as plt
import numpy as np
import gdown

# Download the audio file
file_id = "1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL"
url = f"https://drive.google.com/uc?id={file_id}"
output = "group14.wav"
gdown.download(url, output, quiet=False)

# Load and resample
waveform, sr = torchaudio.load(output)
target_sr = 44100
if sr != target_sr:
    waveform = T.Resample(sr, target_sr)(waveform)
    sr = target_sr
waveform_np = waveform.squeeze().numpy()

# Use only a short segment for pitch estimation
segment = waveform_np[:sr]  # 1 second

# FFT
n = len(segment)
fft_result = np.fft.fft(segment)
fft_freqs = np.fft.fftfreq(n, d=1/sr)
magnitude = np.abs(fft_result[:n // 2])
freqs = fft_freqs[:n // 2]

# Get dominant frequency
peak_index = np.argmax(magnitude)
dominant_freq = freqs[peak_index]

# ---------------------------------------------
# Map to closest musical note
def freq_to_note_name(frequency):
    A4 = 440.0
    if frequency == 0:
```

```
        return "Silence", 0, 0
    # Calculate number of semitones from A4
    semitones_from_A4 = 12 * np.log2(frequency / A4)
    rounded = int(round(semitones_from_A4))
    closest_freq = A4 * (2 ** (rounded / 12))
    offset_cents = 100 * (semitones_from_A4 - rounded)

    # Map index to note name
    note_names = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']
    note_index = (rounded + 9) % 12  # A4 is index 9
    octave = 4 + ((rounded + 9) // 12)

    return f"{note_names[note_index]}{octave}", closest_freq, offset_cents

note_name, ideal_freq, offset = freq_to_note_name(dominant_freq)

# ----------------------------------------------
# Output
print(f"Dominant Frequency: {dominant_freq:.2f} Hz")
print(f"Closest Note: {note_name}")
print(f"Note Frequency: {ideal_freq:.2f} Hz")
print(f"Offset: {offset:+.2f} cents")
```

```
Downloading...
From: https://drive.google.com/uc?id=1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL
To: /content/gdrive/MyDrive/Colab Notebooks/group14.wav
  0%|          | 0.00/232k [00:00<?, ?B/s]
100%|██████████| 232k/232k [00:00<00:00, 39.7MB/s]
Dominant Frequency: 249.00 Hz
Closest Note: B3
Note Frequency: 246.94 Hz
Offset: +14.37 cents
```

Experiment with diffrent pitch factor

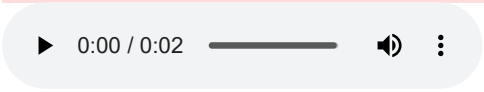
In [4]:

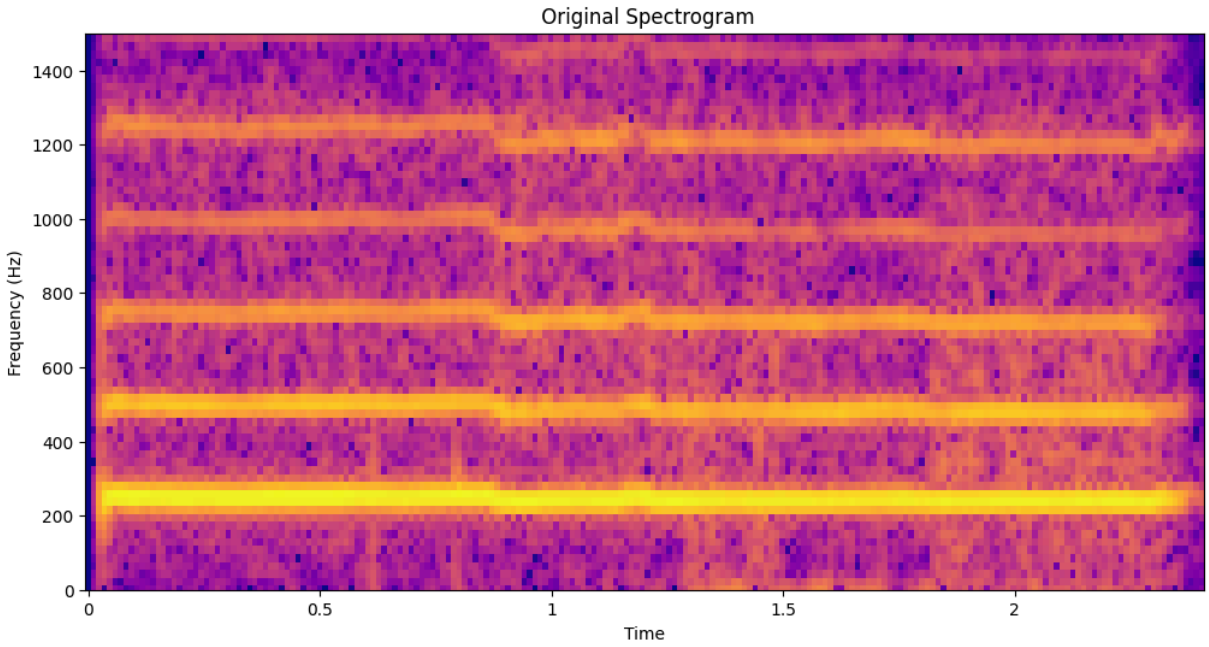
```
# Install dependencies
!pip install torchaudio librosa matplotlib gdown IPython --quiet

# Imports
import torchaudio
import torchaudio.transforms as T
import matplotlib.pyplot as plt
import librosa
import librosa.display
import gdown
import torch
import numpy as np
from IPython.display import Audio, display

# Download audio
file_id = "1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL"
url = f"https://drive.google.com/uc?id={file_id}"
output = "group14.wav"
gdown.download(url, output, quiet=False)

# Load and resample to 44100 Hz
waveform, original_sr = torchaudio.load(output)
target_sr = 44100
if original_sr != target_sr:
    resample = T.Resample(orig_freq=original_sr, new_freq=target_sr)
    waveform = resample(waveform)

waveform_np = waveform.squeeze().numpy()

# Play original audio
print(" Original Audio")
display(Audio(waveform_np, rate=target_sr))

# Spectrogram of original
plt.figure(figsize=(12, 6))
D = librosa.amplitude_to_db(np.abs(librosa.stft(waveform_np)), ref=np.max)
librosa.display.specshow(D, sr=target_sr, y_axis='linear', x_axis='time', cmap='plasma')
plt.ylim(0, 1500)
```

```
plt.yticks(np.arange(0, 1501, 200))
plt.title(" Original Spectrogram")
plt.ylabel("Frequency (Hz)")
plt.show()

# Define pitch factors to test
pitch_factors = [0.8, 0.9, 1.5, 2.1, 2.7, 3.57]
pitch_shifted_versions = {}

# Apply pitch shifting
for factor in pitch_factors:
    n_steps = 12 * np.log2(factor)
    shifted_audio = librosa.effects.pitch_shift(waveform_np, sr=target_sr, n_steps=n_steps)
    pitch_shifted_versions[factor] = shifted_audio

# Plot and play pitch-shifted versions
for factor, audio in pitch_shifted_versions.items():
    print(f"\n Pitch-Corrected Audio (Pitch Factor: {factor:.2f})")
    display(Audio(audio, rate=target_sr))

    plt.figure(figsize=(12, 6))
    D = librosa.amplitude_to_db(np.abs(librosa.stft(audio)), ref=np.max)
    librosa.display.specshow(D, sr=target_sr, y_axis='linear', x_axis='time', cmap='plasma')
    plt.xlim(0,2)
    plt.ylim(0, 1000)
    plt.yticks(np.arange(0, 1501, 200))
    plt.title(f" Spectrogram - Pitch Factor: {factor:.2f}")
    plt.ylabel("Frequency (Hz)")
    plt.tight_layout()
    plt.show()
```

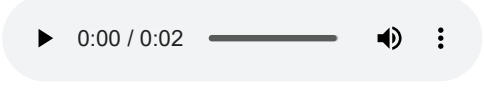
```
Downloading...
From: https://drive.google.com/uc?id=1uZAcg0TOo27o-x5n9pMdrt5-AeeJsiQL
To: /content/gdrive/MyDrive/Colab Notebooks/group14.wav
  0%|          | 0.00/232k [00:00<?, ?B/s]
100%|██████████| 232k/232k [00:00<00:00, 39.3MB/s]
```

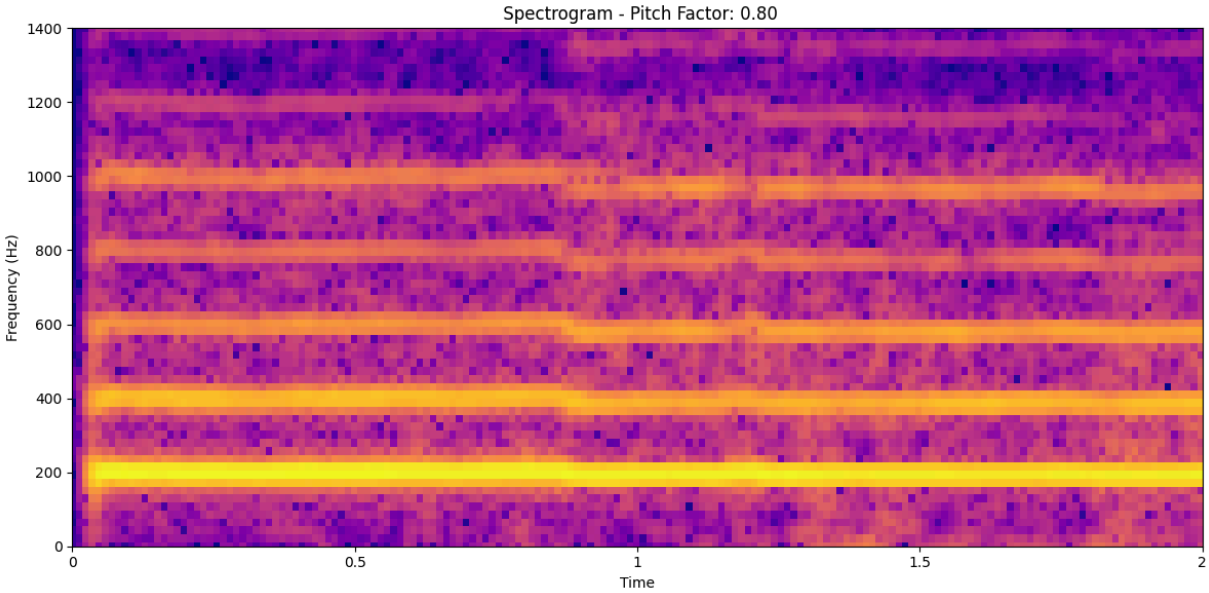
Original Audio





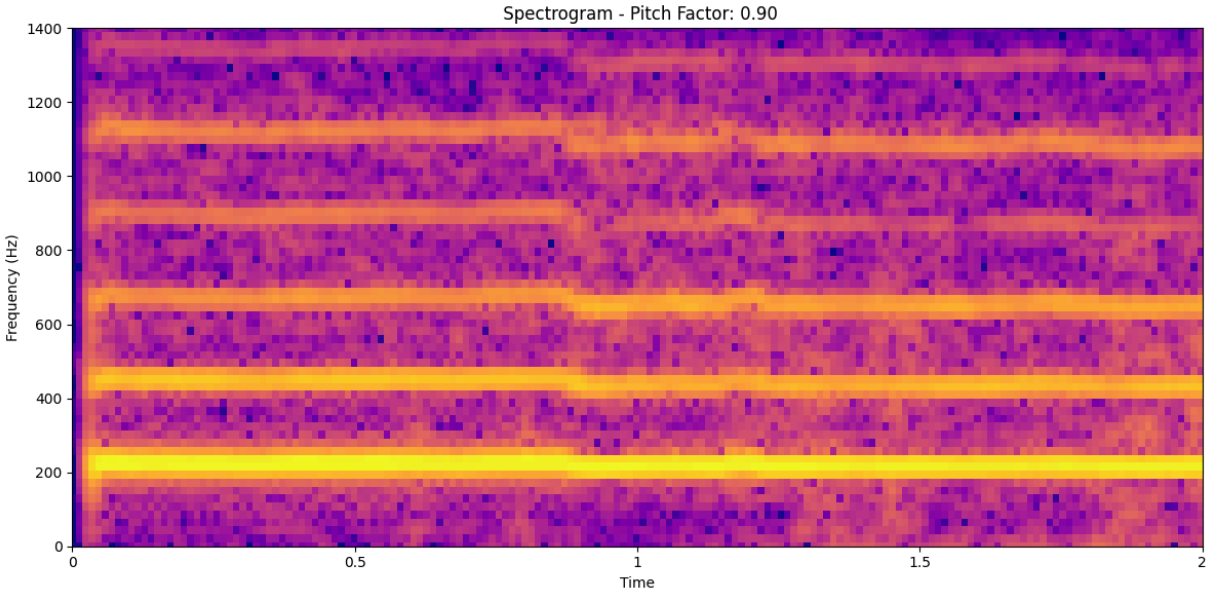
Pitch-Corrected Audio (Pitch Factor: 0.80)





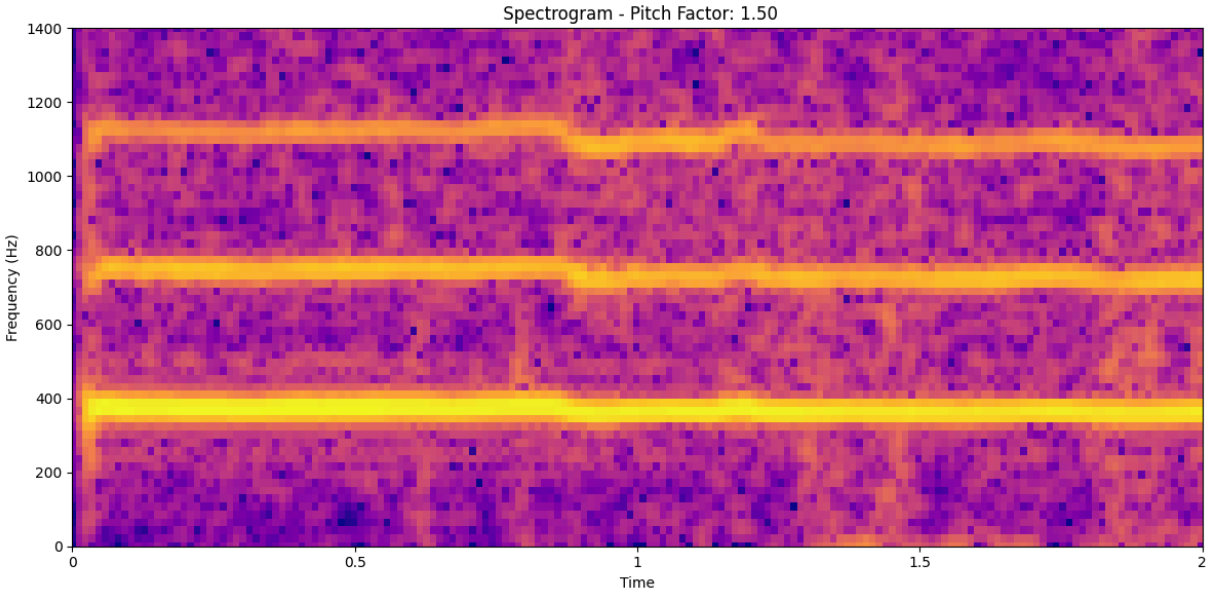
Pitch-Corrected Audio (Pitch Factor: 0.90)

0:00 / 0:02



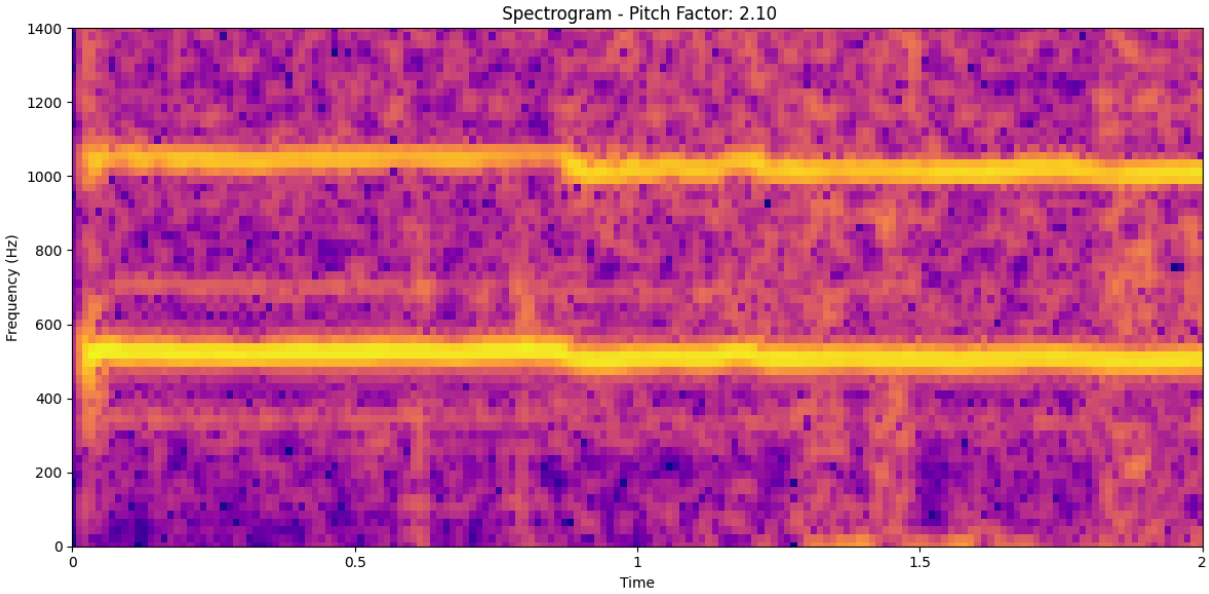
Pitch-Corrected Audio (Pitch Factor: 1.50)

0:00 / 0:02



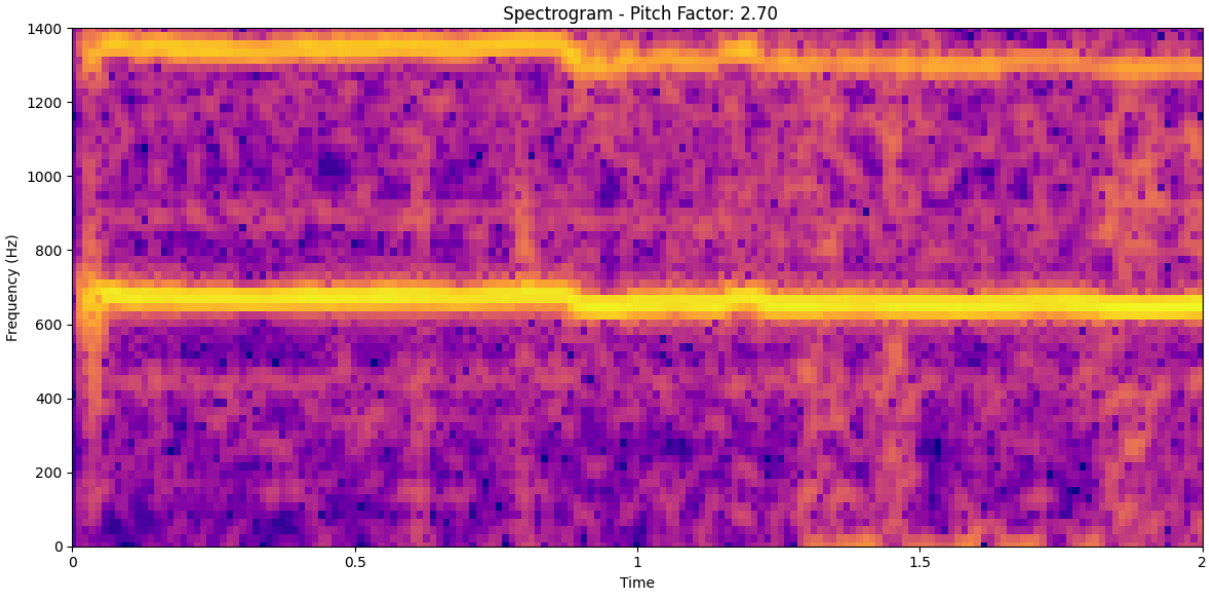
Pitch-Corrected Audio (Pitch Factor: 2.10)

0:00 / 0:02



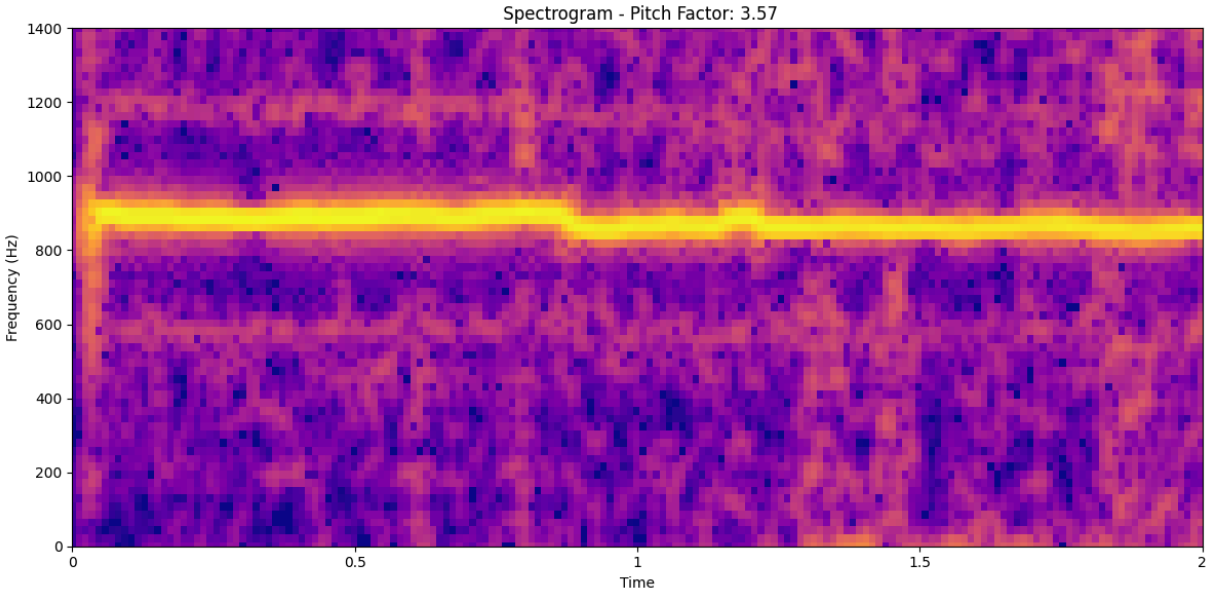
Pitch-Corrected Audio (Pitch Factor: 2.70)

0:00 / 0:02



Pitch-Corrected Audio (Pitch Factor: 3.57)

0:00 / 0:02



Experiment with Pitch Correction Settings

- *Objective* :

Pitch correction was applied using a resampling-based method to adjust vocal recordings toward a standard tuning reference of 440 Hz.

- *Pitch Shift Factors Tested* :

The following pitch shift factors were evaluated: 0.80, 0.90, 1.50, 2.10, 2.70, and 3.57, representing a wide range of pitch modifications.

- *Effective Range Identified* :

The most perceptually and spectrally favorable results were observed in the 0.89 to 1.12 range.

Within this range, audio retained natural vocal timbre and demonstrated improved pitch accuracy.

- *Optimal Shift Factor* :

A shift factor of 1.10 produced the most balanced results.

It aligned the fundamental frequency with musical note B3 (approx. 246 Hz) while maintaining harmonic clarity.

Impact of Extreme Shift Factors:

- Pitch factor 1.10 produced the most balanced correction—pitch was closer to (B3 ~ 246 Hz), retained a naturalness and made it musical.
- Factors 0.80 and 1.20 resulted in more obvious artifacts (e.g., "metallic" quality, muffled or synthetic-sounding harmonics).
- Factor 1.00 served as the baseline for comparison.
- When the pitch factor is greater than 1.0 (e.g., 1.5, 2.10,3.57), the frequency components in the spectrogram shift upward. This indicates that the pitch of the audio has increased, with harmonics and overtones appearing at higher frequencies than in the original.
- Conversely, when the pitch factor is less than 1.0 (e.g., 0.90, 0.80), the frequency components shift downward in the spectrogram, reflecting a lower pitch. This downward shift compresses the harmonic content into a lower frequency range.
- More extreme shifts (e.g., 0.80, 1.50, 2.10, 2.70, 3.57) introduced audio artifacts, such as:

  1.Metallic resonances

  2.Spectral smearing

  3.Degradation in harmonic structure

-These effects were confirmed through spectrogram and FFT analysis.

-The artifacts degraded both technical fidelity and perceptual quality.

Limitations of pitch correction.

- The Phase Vocoder algorithm works best on monophonic, steady-pitch sources. If vibrato or vocal fluctuations are present, it can distort the sound.
- Time-domain features (e.g., articulation, consonants) are often blurred because the algorithm operates in the frequency domain.
- Formant preservation is not always guaranteed, in the naive pitch shifting, which can lead to unnaturalness. *Pitch shifting, which alters the perceived pitch of a sound, is primarily achieved by changing the frequency content of the audio signal, not by introducing a phase shift which is one of drawback of pitch correction. [1]

Artifacts of pitch correction.

[1]

Artifact Type	Description
Robotic sound	Caused by loss of natural pitch modulations
Smearing	Transients and consonants became less sharp
Formant distortion	Atypical vocal quality, especially with large pitch shifts
Phase artifacts	Wavy or "watery" effect due to imperfect phase reconstruction

References

1. ^Jeon, Hohyub & Jung, Yongchul & Lee, Seongjoo. (2020). Area-Efficient Short-Time Fourier Transform Processor for Time–Frequency Analysis of Non-Stationary Signals. Applied Sciences. 10. 7208. 10.3390/app10207208.

2. ^ Gupta, Gaurav & Zhong, Ming & Gholami, Shahrzad & Lavista Ferres, Juan. (2021). Comparing recurrent convolutional neural networks for large scale bird species classification. Scientific Reports. 11. 10.1038/s41598-021-96446-w.

3. ^ Richter, Yair & Balal, Nezah & Pinhasi, Yosef. (2023). Neural-Network-Based Target Classification and Range Detection by CW MMW Radar. Remote Sensing. 15. 10.3390/rs15184553.

4. ^ Archer, Martin & Cottingham, Marek & Hartinger, Michael & Shi, Xueling & Coyle, Shane & Hill, Ethan & Fox, Michael & Masongsong, Emmanuel. (2022). Listening to the Magnetosphere: How Best to Make ULF Waves Audible. Frontiers in Astronomy and Space Sciences. 9. 877172. 10.3389/fspas.2022.877172.

5. ^Oppenheim, A. & Schafer, R. (2013). Discrete-Time Signal Processing. (3rd ed.). Pearson International (Section 10.5). https://elibrary.pearson.de/book/99.150005/9781292038155

In [5]:
```
##########################################################
# class_3_group_14

import os
from google.colab import drive, files

# Attempt to mount Google Drive
try:
    drive.mount('/content/gdrive/')
    print("Google Drive mounted successfully!")
except Exception as e:
    print(f"Error mounting Google Drive: {e}")
    print("Please try running the cell again or check your Google Drive connection.")

# Your copy is typically saved in this folder
filename = 'Another copy of Class_3_group_14'
path_to_folder = '/content/gdrive/MyDrive/Colab Notebooks/'

file_path = os.path.join(path_to_folder, filename + '.ipynb')
html_path = os.path.join(path_to_folder, filename + '.html')

# Check if the notebook file exists before attempting to convert
if os.path.exists(file_path):
    print(f"Notebook found at: {file_path}")
    !jupyter nbconvert --to html "{file_path}"
    # Check if the HTML file was created before attempting to download
    if os.path.exists(html_path):
        print(f"HTML file created at: {html_path}")
        files.download(html_path)
    else:
        print(f"Error: HTML file not created at {html_path}")
else:
    print(f"Error: Notebook not found at {file_path}. Please ensure the filename and path are correct.")
##########################################################
```

```
Drive already mounted at /content/gdrive/; to attempt to forcibly remount, call drive.mount("/content/gdr
ive/", force_remount=True).
Google Drive mounted successfully!
Notebook found at: /content/gdrive/MyDrive/Colab Notebooks/Another copy of Class_3_group_14.ipynb
[NbConvertApp] Converting notebook /content/gdrive/MyDrive/Colab Notebooks/Another copy of Class_3_group_
14.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 2 image(s).
[NbConvertApp] Writing 450649 bytes to /content/gdrive/MyDrive/Colab Notebooks/Another copy of Class_3_gr
oup_14.html
HTML file created at: /content/gdrive/MyDrive/Colab Notebooks/Another copy of Class_3_group_14.html
```